

포인터와 스택 개념 정리

1. 포인터 개념

1.1 포인터의 정의

- 포인터란 다른 변수의 주소를 저장하는 변수이다.
- 메모리는 주소(Address)와 값(Value)으로 구성된다.
- 변수의 주소를 통해 값을 간접적으로 참조(Indirect Reference)할 수 있다.

1.2 포인터 선언 및 사용

```
int a = 10;        // 변수 선언 및 초기화
int *p = &a;       // 포인터 선언 및 초기화 (a의 주소 저장)
```

- *p 는 a를 간접 참조하여 값을 가져온다.
- &a 는 a의 메모리 주소를 반환한다.

1.3 포인터 연산자

- &: 주소 연산자 → 변수의 주소를 반환
- *: 간접 참조 연산자 → 포인터가 가리키는 주소의 값을 반환 또는 수정

예제 코드

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a; // a의 주소 저장

    printf("변수 a의 값: %d\n", a);
    printf("포인터 p가 가리키는 값: %d\n", *p);

    *p = 20; // 간접 참조를 통해 a 값 변경
    printf("변경된 변수 a의 값: %d\n", a);

    return 0;
}
```

1.4 포인터의 중요성

- 메모리 효율성: 직접적인 메모리 접근 가능
- 함수 매개변수 전달: Call by Reference 방식 지원
- 동적 메모리 할당: malloc()과 같은 함수 사용 가능

2. 포인터의 다양한 형태

2.1 포인터의 포인터 (Double Pointer)

```
int a = 10;
int *p = &a; // a의 주소 저장
int **pp = &p; // p의 주소 저장

printf("a의 값: %d\n", **pp); // 10
```

- `**pp` 를 사용하면 `a`의 값을 참조 가능

2.2 Void 포인터 (Generic Pointer)

```
void *ptr;
int a = 10;
ptr = &a;
printf("값: %d\n", *(int*)ptr); // 형 변환 후 참조
```

2.3 함수 포인터

```
#include <stdio.h>

void func() {
    printf("Hello, Function Pointer!\n");
}

int main() {
    void (*fp)() = func; // 함수 포인터 선언
    fp(); // 함수 호출
    return 0;
}
```

3. 동적 메모리 할당

3.1 malloc() 함수 사용

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int*)malloc(sizeof(int)); // 정수형 동적 할당
    *p = 100;
}
```

```
printf("값: %d\n", *p);
free(p); // 메모리 해제
return 0;
}
```

3.2 calloc() 함수 사용

```
int *arr = (int*)calloc(5, sizeof(int));
```

- **calloc()**은 할당된 메모리를 0으로 초기화함

3.3 realloc() 함수 사용

```
arr = (int*)realloc(arr, 10 * sizeof(int));
```

- 메모리 크기 재조정 가능

4. 스택(Stack)

4.1 스택의 개념

- **LIFO (Last In First Out)** 구조
- 가장 나중에 들어온 데이터가 가장 먼저 나감

4.2 스택의 연산

- **Push**: 데이터 삽입
- **Pop**: 데이터 삭제
- **Peek**: 최상단 데이터 확인

4.3 스택 구현 (배열 기반)

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

void push(int val) {
    if (top >= MAX - 1) {
        printf("스택 오버플로우\n");
        return;
    }
    stack[++top] = val;
}
```

```
int pop() {
    if (top < 0) {
        printf("스택 언더플로우\n");
        return -1;
    }
    return stack[top--];
}

int main() {
    push(10);
    push(20);
    printf("pop: %d\n", pop());
    return 0;
}
```

4.4 동적 스택 구현 (링크드 리스트 기반)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* top = NULL;

void push(int val) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = val;
    newNode->next = top;
    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("스택 언더플로우\n");
        return -1;
    }
    int val = top->data;
    Node* temp = top;
    top = top->next;
    free(temp);
    return val;
}

int main() {
    push(10);
    push(20);
    printf("pop: %d\n", pop());
}
```

```
    return 0;  
}
```

5. 연산자 우선순위

5.1 연산자 우선순위 정리

1. 괄호 `()`
2. 증감 연산 `++ --`
3. 산술 연산 `* / % + -`
4. 관계 연산 `< > <= >= == !=`
5. 논리 연산 `&& ||`
6. 대입 연산 `= += -= *= /=`

6. 정리

- 포인터는 메모리 주소를 저장하고 간접 참조할 수 있는 강력한 도구
- 동적 메모리 할당은 `malloc()`, `calloc()`, `realloc()`, `free()` 등을 통해 관리
- 스택(Stack)은 후입선출(LIFO) 구조로 동작하며, 배열과 링크드 리스트로 구현 가능
- 연산자 우선순위를 이해하고 적절한 괄호 사용이 중요

실습을 통해 개념을 확실히 익히는 것이 중요!